

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

There are $n + 1$ agents $a_0, \dots, a_n$, with a_0 being Pac-Man and $a_1, \dots, a_n$ the ghosts. A depth is one move from every agent, so d only drops once the last ghost a_n has moved.

$$V_{\text{minimax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{if } \text{IsEnd}(s) \\ \text{Eval}(s) & \text{if } d = 0 \\ \max_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s, a), d) & \text{if } \text{Player}(s) = a_0 \\ \min_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s, a), d) & \text{if } \text{Player}(s) = a_i, 1 \leq i < n \\ \min_{a \in \text{Actions}(s)} V_{\text{minimax}}(\text{Succ}(s, a), d - 1) & \text{if } \text{Player}(s) = a_n \end{cases}$$

The first two cases are the two ways to stop: the game has actually ended, or the depth budget has run out and the evaluation function stands in for the rest of the game. The last three recurse, with Pac-Man taking the max and the ghosts the min, and only the last ghost's line passing $d - 1$ down, so that a full round of moves costs one depth rather than one per agent.

3.a

$$V_{\text{exptmax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{if } \text{IsEnd}(s) \\ \text{Eval}(s) & \text{if } d = 0 \\ \max_{a \in \text{Actions}(s)} V_{\text{exptmax}}(\text{Succ}(s, a), d) & \text{if } \text{Player}(s) = a_0 \\ \frac{1}{|\text{Actions}(s)|} \sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}(\text{Succ}(s, a), d) & \text{if } \text{Player}(s) = a_i, 1 \leq i < n \\ \frac{1}{|\text{Actions}(s)|} \sum_{a \in \text{Actions}(s)} V_{\text{exptmax}}(\text{Succ}(s, a), d - 1) & \text{if } \text{Player}(s) = a_n \end{cases}$$

Same as 1a except the ghosts take the average of their successors instead of the min, since each ghost picks every legal move with the same probability $1/|\text{Actions}(s)|$. Pac-Man still takes the max, and d still only drops after the last ghost moves.

5.a

Minimax assumes the ghosts play optimally against Pac-Man, actively coming for it to minimize its score, so in `trappedClassic` it believes it is doomed to lose whatever it does, and the best it can do is cut the time penalty by dying as fast as possible, which means running into the closest ghost. Expectimax instead assumes the ghosts move uniformly at random, so they are not actively pursuing Pac-Man and there is a real chance they wander off, which makes going for the pellets worth more in expectation than a certain early death.

5.b

Change how the score is computed so that it rewards each move Pac-Man stays alive instead of penalizing it, while keeping pellets worth more than the survival reward so it still goes after them rather than hiding. A flat penalty for being eaten would not help here, since minimax believes it gets eaten in every line of play and so every option pays the same penalty; what differs is how soon it happens, and with a reward for surviving, dying later scores higher than dying sooner, so minimax runs from the ghosts and buys time instead of charging the closest one, giving the real, random ghosts a chance to wander off as expectimax expects.

5.c

A racing game agent is rewarded for reaching the finish line in the minimum time, on a circuit where the start line is also the finish line. Instead of driving the lap, the agent turns around and crosses the line backwards straight away, which reaches the finish line faster than any real lap could. This is reward hacking: the agent fully satisfies the objective as written, since it does reach the finish line in minimal time, but not the designer's actual goal of completing the course, because the objective only checks that the line was crossed and not that the lap was driven.

6.a

Claude Code's data page (code.claude.com/docs/en/data-usage) says consumer accounts (Free, Pro and Max) are used for training when the model improvement setting is on, with data kept for five years instead of 30 days, while commercial accounts are not trained on, and usage metrics and error reports go to third-party services but never include code or prompts. The page is easy to follow, but it only turns up if you search for Claude Code's policy specifically, since the general privacy and usage pages do not point to it. What concerns me is that none of Anthropic's official pages say whether training is on by default, so whether a user's sessions are kept for 30 days or five years and used to train future models depends on a setting whose starting state is never stated. Upstream, the Opus 5 system card only says the model was trained on a "proprietary mix" of public internet data, public and private datasets and synthetic data, with no list of what those are and no mention of user data, even though the data page says consumer sessions do feed training when the setting is on.

6.b

Some things clearly aim to build trust: the Claude Code data page gives exact retention periods and an environment variable to switch off each kind of telemetry, the training setting can be changed at any time, and the October 2026 usage policy update (anthropic.com/news/2026-usage-policy-update) is written in plain language and explains why each rule changed. What undermines it is the training default never being stated, turning the setting off only excluding new sessions so anything already trained on stays in existing models, and the data page only turning up if you search for Claude Code specifically. A simple fix would be to show the current training setting, and what it means for retention, the first time Claude Code is launched, so the choice is visible rather than buried in settings. On the openness spectrum Claude Code sits at the closed end, with no released weights or training data, but Anthropic does publish a system card for each model, its usage policy and regular threat intelligence reports. That documentation lets outsiders check some of the safety claims, but because the weights and data are closed, the rest still has to be taken on trust, which limits how accountable the company can really be held.

6.c

- **Fairness** (demographic parity). Each system card runs a demographic bias benchmark (BBQ) and a political evenhandedness test, but only on the chat model, nothing for Claude Code, and it admits output quality varies by language without saying by how much. They could publish Claude Code results per language.
- **Transparency** (publishing model cards). Anthropic publishes one for every model, but it gives few details on the model itself and only calls the data a "proprietary mix".
- **Accountability** (users can report and contest outputs). There is `/bug`, `/feedback` and support, but it is private and asynchronous, with no public record of what was reported or fixed. A public log of reported issues and their outcomes would help.

6.d

Users can export their data (Settings, Privacy, Export data), switch model training on or off, delete individual cloud sessions, turn off each kind of telemetry with an environment variable, and report problems through `/bug` and `/feedback`. But reporting has a cost: transcripts sent through `/feedback`, `/bug` or `/share`, including code, are kept for five years, even for a user who has training switched off and would otherwise get 30 days. A useful addition would be a per-session view showing whether each session was kept for training and when it will be deleted, so users can check the privacy promises themselves instead of relying on a setting whose default is not stated.

6.e

Anthropic paid \$1.5 billion to settle *Bartz v. Anthropic*, approved in July 2026, over training on pirated books, though the settlement only covers past conduct and not Claude's outputs. Music publishers are also suing over song lyrics in Claude's training and outputs, including a 2026 suit over more than 20,000 songs allegedly torrented from pirate libraries. Under the commercial terms, customers own their outputs and Anthropic will defend them against third-party IP claims, but under the consumer terms covering Free, Pro and Max, users own their outputs with no indemnity at all and must instead indemnify Anthropic. So if Claude Code reproduced someone else's licensed code in a personal project, a paying business would be protected while an individual user would carry the risk themselves.